DecL Cheat Sheet (1/3): agg and pnl

An Aggregate compound distribution has the clause structure:

agg <NAME> <EXPOSURE> <LAYERS*> <SEVERITY> <OCC_RE*> <FREQUENCY> <AGG_RE*> <APPROX*> <TRAILER*>

Key: <INPUT> user input; lower_case a DecL keyword; CLAUSE a sub-clause; options a|b|c; inf infinity; * optional. Build with build('agg ...').

1. Name & references

agg <NAME> ...
sev <NAME> <SEVERITY> *standalone severity*

Names match [a-zA-Z][._:~a-zA-Z0-9-]*.
Reference stored objects: agg.<NAME>, sev.<NAME>, dist.<NAME>; clone with agg <NEW> agg.<OLD>.

2. Exposure

<EL> loss
<PREMIUM> premium at <LR> lr
<EXPOSURE> exposure at <RATE> rate
<CLAIMS> claims
dfreq <OUTCOMES> <PROBS*> *fixed empirical freq*

Outcomes [1 2 3 4] or [2:10:2]; probs [.5 .25 .25] or omitted = equally likely.

3. Limit / layers (optional)

<LIMIT> xs <ATTACHMENT>
tower [<BREAKS>]

Occurrence limits on ground-up severity, unlimited reinstatements, losses conditional on attaching by default.

4. Severity (continuous)

sev <DIST> <MEAN> cv <CV>
sev <DIST> <SHAPE1> <SHAPE2*>
sev sev.<NAME> *built-in*

<SCALE> * SEV + <LOC> *scale & shift*
SEV wts [<W>] *mixture*
SEV splice [<LB> <UB>] *conditional in layer*
SEV ! *unconditional (when* ATTACH>0)
ssev ... *spliced severity*

5. Severity (discrete)

dsev <OUTCOMES> <PROBS*> *discrete*
<DIST> xps [<X>] [<P>] *empirical points*
picks [<X>] [<P>] *match layer picks*

dsev [1:6] is a fair die; xps attaches outcomes/probabilities to a parametric severity.

6. Frequency

poisson, bernoulli, fixed, geometric, logarithmic,
binomial <P>, negbin <VAR_MULT>, neyman <CLAIMS/OCC>, pascal <CV> <CLAIMS/OCC>

mixed <MIX> <SHAPE1> <SHAPE2*>
MIX = gamma|delaporte|ig|sig|sichel|beta

FREQ zt *zero-truncated*
FREQ zm <P0> *zero-modified,* $\Pr(N=0)=p_0$

7. Occurrence reinsurance

occurrence net of LAYERS
occurrence ceded to LAYERS

LAYER = <LIMIT> xs <ATTACH>
  | <SHARE> so <LIMIT> xs <ATTACH>
  | <PCT> po <LIMIT> xs <ATTACH>
LAYERS = LAYER and LAYER ... *or* tower [...]

so = share of, po = part of; $0 \le \text{share} \le 1$.

8. Aggregate reinsurance

aggregate net of LAYERS
aggregate ceded to LAYERS
aggregate (net of|ceded to) tower [<BREAKS>]

Same LAYER syntax as occurrence, applied to the aggregate loss.

9. Approximate (optional)

approximate exact *(default)*
approximate sgamma *shifted gamma*
approximate slognorm *shifted lognormal*

Alias approx. Replaces freq × sev convolution with a single severity matched to the first three aggregate moments.

10. PnL (premium – loss)

pnl <NAME> <PREMIUM> prem - <AGG-BODY>

Profit-and-loss = premium minus loss. The premium is one deterministic amount subtracted once *for the book (aggregate-level shift), unlike a constant in sev (per claim). The body after prem - is an ordinary aggregate; a bare lr binds to the stated premium.*

pnl P 1000 prem - 700 loss sev lognorm 50 cv 2 poisson

11. Trailer, vectors, math

Trailer note{free text}, hints{bs=100; log2=17} *(build settings; both optional, order-free)*

Vectorize [1 2 3] claims; [100 200] premium at [.8 .7] lr; [250 250 500] xs [0 250 500] *zip*; [1 3] * lognorm [.7 1.5] wts [.6 .4]. *Vectors broadcast; layers / exposure zip.*

Math 1/2, 3**4, exp(2), 12.4%. *Warning: minus binds to the number,* $-4^2 = 16$ *(no unary minus).*

aggregate v.1.0.0a67
ease | confidence | precision

2026-06-18 21:11

DecL Cheat Sheet (2/3): port, mv, netceded

These declarations build collections and couplings of aggregates. Each component is an ordinary agg (or pnl) declaration.

1. Portfolio (port)

```
port <NAME> <TRAILER*>
  agg <UNIT1> ...
  agg <UNIT2> ...
  ...
```

A Portfolio is a list of independent units sharing a discretization grid. Units may be agg or pnl lines, or references agg.<NAME>.

2. port example

```
port MyBook
  agg A 100 claims sev lognorm 50 cv 2
poisson
  agg B 200 claims sev gamma 30 cv 1 poisson
```

Units are added independently; the portfolio total is their convolution. Per-unit reinsurance is set on each agg line.

3. Multivariate (mv)

```
mv <NAME> <SHARED-EXPOSURE>
  agg <A> ...
  agg <B> ...
  copula <KIND> <P*>
  <FREQUENCY*>
```

Alias multivariate. Couples exactly two *component severities by a copula and accumulates them with a* shared *outer frequency via a 2D FFT. Frequency optional (default poisson).*

4. Copula clause

copula <KIND> <P> *one-parameter*
copula <KIND> *parameter-free*
omitted = copula independent

```
KIND = normal|gumbel|clayton|fgm|independent
```

```
mv M 100 claims agg A ...  agg B ...  copula
gumbel 0.4
```

5. netceded

```
netceded <AGG>
```

Build the bivariate (net, ceded) joint outcome of one reinsured aggregate — the two coupled marginals are the net *and* ceded *positions of the same loss, so their dependence is exact (comonotone within the layer). Lets you see net and ceded* jointly, *not just their marginals.*

```
netceded agg G 100 claims sev lognorm 50 cv
2 occurrence net of 500 xs 500 poisson
```

DecL Cheat Sheet (3/3): `distortion`

A distortion g defines a coherent risk measure / pricing functional. Declare one by kind and its natural parameters; the kind → parameter mapping lives on each `Distortion` subclass.

1. Declaration

```
distortion <NAME> <KIND> <PARAMS>
```

Alias `dist`*.* <PARAMS> *is a flat list of the kind's natural parameters (see table). Reference a stored distortion as* `dist.`<NAME>*.*

```
distortion D1 ph 0.9
dist D2 wang 0.25
dist D3 power 0 100 2
```

2. Kinds & parameters

`ph` <a> *proportional hazard*
`wang` <lam> *Wang / normal shift*
`dual` <b> *dual moment*
`tvar` <p> *tail VaR at p*
`ccoc` <r> *constant cost of capital*
`bitvar` <p0> <p1> <w1> *two-point*
`beta` <a> <b>
`power` <x0> <x1> <alpha>

3. More kinds

`wtdtvar ...` *weighted TVaR (knots)*
`cll` <b> *capped log-linear*
`clin` <slope> *capped linear*
`lep` <r> *leverage-equivalent pricing*
`ly` <r> *linear yield*

List available kinds with `Distortion.available_distortions()`.

4. Combinators

```
distortion <NAME> minimum dist.<A> dist.<B>
...
distortion <NAME> mixture dist.<A> dist.<B>
...  wts [<W>]
```

`minimum` = *pointwise min of the referenced distortions;* `mixture` = *weighted average (weights via* `wts`*).*

5. Usage

Apply a distortion to price an aggregate or portfolio:

```
a = build('agg A 100 claims sev lognorm 50
cv 2 poisson')
d = build('distortion D ph 0.9')
a.price(0.99, d)
```

or reference it inside pricing calls by `dist.`<NAME>*. A distortion satisfies* $g(0) = 0$*,* $g(1) = 1$*, increasing and concave.*